Progressive Education Society's
MODERN COLLEGE OF ENGINEERING, Pune -05.
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)
MCA Department

PRACTICAL SUBMISSION RECORD- A.Y. 2024-25

Class: SYMCA **Div: B** **Semester: III**	**Course Code: MCA01602** **Course Name: Data Science Laboratory**	**Batch: S2**
Name: Ajit Bajarang Mali	**Roll No: 52140**	
CO No: CO602.4	**Assignment No: 13**	

1. Introduction

The "Smart Digit Teaching system" is an interactive practical project for the SYMCA course. It is an educational web application built using Python, Streamlit, and TensorFlow.

The system is designed to help users practice drawing digits (0-9). It provides a target number via audio, captures the user's drawing on a digital canvas, and then uses a custom-trained Convolutional Neural Network (CNN) to grade the drawing. The application provides an immediate numerical score and qualitative feedback to help the user learn and improve their writing.

2. Objective

The primary objective of this project is to develop an AI-powered grading system that:

- **Presents a Task:** Asks the user to draw a specific, randomly selected digit.
- **Provides Audio Guidance:** Uses Google Text-to-Speech (gTTS) to play an audio prompt of the target number.
- **Captures Input:** Allows the user to draw the digit on an interactive canvas.
- **Analyzes Drawing:** Preprocesses the drawn image and feeds it into a trained CNN model for classification.
- **Delivers Feedback:** Calculates a score based on the model's confidence and provides constructive, text-based feedback to the user (e.g., "Perfect Match!", "Mistake! I saw a...").

3. Method and Materials

This project integrates a machine learning backend with a web-based frontend.

Materials (Software & Libraries)

- **Dataset:** MNIST dataset for training the digit recognizer.
- **Machine Learning:** tensorflow (Keras): For building, training, and running the CNN model.
- **Application & UI:** streamlit: The core framework for building the interactive web app. streamlit-drawable-canvas: The specific component used to create the drawing area.
- **Data Processing & Utilities:** opencv-python (cv2): Used extensively for preprocessing the user's drawing (e.g., grayscale conversion, thresholding, finding contours, and resizing). numpy: For numerical operations on image arrays.
- **Audio:** gTTS: To generate the text-to-speech audio files for the target numbers.

Method (CNN Model Architecture)

The core of the system is a Convolutional Neural Network (CNN) defined using tensorflow.keras.models.Sequential. The architecture is as follows:

1. **Input Layer:** Implicitly handles the $28 \times 28 \times 1$ input images.
2. **Conv2D Layer 1:** 32 filters, (3,3) kernel, ReLU activation.
3. **MaxPooling2D Layer 1:** (2,2) pool size
4. **Conv2D Layer 2:** 64 filters, (3,3) kernel, ReLU activation.
5. **MaxPooling2D Layer 2:** (2,2) pool size.
6. **Flatten Layer:** Converts the 2D feature maps into a 1D vector.
7. **Dropout Layer:** A Dropout of 0.25 is applied to reduce overfitting.
8. **Dense Layer (Hidden):** 128 neurons, ReLU activation.
9. **Dense Layer (Output):** 10 neurons (one for each digit, 0-9), softmax activation.

4. Procedure

The project is executed in two main phases: model training and application deployment.

A. Model Training (train_model.py)

1. **Load Data:** The MNIST dataset (training and test sets) is loaded
2. **Preprocess Data:** The image data is reshaped to ($28, 28, 1$), normalized (divided by 255), and labels are one-hot encoded (to_categorical)
3. **Define Model:** The CNN architecture (described in Method) is defined
4. **Compile & Train:** The model is compiled and then trained on the training data for 10 epochs with a batch size of 128
5. **Save Model:** The fully trained model is saved to a file named drawing_grader_cnn.h5.

B. Application Workflow (app.py)

1. **Initialization:** The Streamlit app loads the saved drawing_grader_cnn.h5 model. It initializes a "session state" to keep track of the target_number, score, and feedback_message.
2. **Task Generation:** A random target_number (0-9) is generated. The generate_audio function uses gTTS to create an audio file for this number, which is presented to the user via st.audio.
3. **User Input:** The user draws the digit on the st_canvas component. The canvas is set to a 300x300 size with a black background and white stroke.
4. **Answer Submission:** When the user clicks "Check Answer", the following occurs:
   - **Image Preprocessing:** The preprocess_image function converts the 300x300 canvas data into a $1 \times 28 \times 28 \times 1$ image suitable for the model. This involves:
     - Converting to grayscale
     - Applying a binary threshold
     - Finding the largest contour (the digit)
     - Cropping the image to the digit's bounding box
     - Resizing the cropped image to 20x20 [46]and padding it to 28x28 to center the digit
     - Normalizing (dividing by 255) and reshaping the final array
   - **Prediction:** The processed image is passed to model.predict()
   - **Grading:** The get_grade_and_feedback function analyzes the model's predictions

- It identifies the predicted digit (the one with the highest confidence)
- It compares the predicted digit to the target digit
- If they match, the score is high (e.g., confidence * 100)
- If they don't match, the score is low (e.g., confidence * 50) and the feedback indicates the mistake.

5. **Display Results:** The UI updates to show the numerical score and the text feedback. A debug section also shows the 28x28 processed image and a bar chart of the AI's confidence levels
6. **Next Number:** Clicking "Next Number" picks a new random digit, generates new audio, and clears the canvas by incrementing a counter in its key

5. Result

The output of the project is a functional, real-time "Smart Digit Teaching system"[585858]. The final screenshot [59] demonstrates a successful run where:

- The application presents a target number and an audio prompt
- The user has drawn a '0' on the canvas.
- The "AI's Actual Input" debug view shows the 28×28 processed image that was fed to the model
- The "Confidence Breakdown" bar chart shows a very high probability for the digit '0', confirming the model's high confidence
- The "Your Score" section displays a grade, and the feedback "Perfect Match! The AI is 100.0% sure you drew a 0" is shown